PRIVATE LOGIN

Zero-Knowledge Identity Infrastructure for Midnight

Master Project Documentation

Version 1.0 — December 2025

The "Login with Privacy" Button for Web3

Executive Summary

Private Login is a zero-knowledge identity verification infrastructure built on Midnight. It enables users to prove attributes about themselves (age, residency, token holdings, credentials) without revealing underlying personal data. The system serves both consumers (via a credential wallet) and developers (via an SDK/API), positioning itself as essential infrastructure for the Midnight ecosystem.

The core value proposition is simple: ***"Prove who you are without showing who you are."***

This is the "Stripe for identity" play—every dApp that needs age verification, KYC-lite, token-gating, or credential verification becomes a paying customer. As Midnight grows, Private Login grows with it.

The Problem

For Users

- **Data Over-Exposure:** Current identity systems require sharing full documents (passport, ID, utility bills) when only specific attributes need verification
- **Honeypot Risk:** Centralized identity databases become targets—breaches expose millions of users
- **Repetitive Verification:** Users re-verify identity across dozens of platforms, creating friction and risk
- **No Ownership:** Identity data is controlled by platforms, not users

For Developers

- **Compliance Burden:** Implementing KYC/age verification is expensive and complex
- **Liability Exposure:** Storing user identity data creates legal and security obligations
- **User Friction:** Heavy verification requirements kill conversion rates
- **No Web3-Native Solution:** Most identity solutions are Web2-centric and don't integrate with blockchain architectures

The Solution

Private Login provides two integrated products:

1. Developer SDK & API

A drop-in solution for dApp developers to add identity verification without handling sensitive data:

- **Simple Integration:** Add verification with a few lines of TypeScript
- **Pre-Built Verifiers:** Age (18+, 21+), token holdings, NFT ownership, DAO membership
- **Custom Policies:** Define complex verification rules (e.g., "over 18 AND holds 1000+ tokens")
- **Zero Data Liability:** Developers receive proofs, never raw identity data
- **React Components:** Pre-built UI components for common flows

2. Consumer Credential Wallet

A user-facing application where individuals manage their verifiable credentials:

- **Credential Storage:** Securely store verified attributes locally (age, residency, certifications)
- **Selective Disclosure:** Choose exactly what to prove for each verification request
- **One-Click Verification:** Approve verification requests from integrated dApps
- **Audit Trail:** See history of what was verified, when, and for whom
- **Credential Issuance:** Obtain credentials from trusted issuers (future: government, institutions)

How It Works

The Verification Flow

1. **dApp Requests Verification:** "User must be 18+ to access this market"
2. **User Opens Credential Wallet:** Sees the request and what attribute is needed
3. **User Approves Disclosure:** Selects which credential to use for proof
4. **ZK Proof Generated:** Wallet creates proof that user is 18+ without revealing birthdate
5. **Proof Submitted On-Chain:** Midnight verifies the proof cryptographically
6. **dApp Receives Confirmation:** Boolean result—verified or not—no personal data transmitted

Midnight's Role

Midnight's dual-state architecture is purpose-built for this:

- **Private State:** Stores user credentials encrypted; only the user can access
- **Public State:** Records verification results for dApps to query
- **ZK Proofs:** Compact smart contracts generate proofs that bridge private to public
- **Selective Disclosure:** Users control exactly what gets proven and to whom

Technology Stack

Component	Technology
Smart Contracts	Compact (Midnight's ZK-native language)
SDK	TypeScript, Midnight.js 1.0.0
Credential Wallet	React + Vite, Chrome Extension or Web App
Wallet Integration	Lace, NuFi, Vespr via DApp Connector API v2.0
Proof Server	Midnight Docker images (local dev), hosted service (production)
Identity Standards	W3C Verifiable Credentials (future compatibility)
Network	Midnight Testnet-02 → Mainnet

Development Roadmap

Phase 1: Foundation (Weeks 1-4)

Goal: Master Midnight development and deploy first contracts

1. Set up Midnight development environment (Compact compiler, proof server, VS Code extension)
2. Complete official tutorials (counter, bulletin board)
3. Study existing Midnight identity projects (IAMX partnership patterns)
4. Deploy test contracts demonstrating private/public state separation
5. Document learnings and architecture decisions

Deliverable: Working dev environment, first contracts on testnet

Phase 2: Core SDK MVP (Weeks 5-10)

Goal: Build minimum viable SDK with one verification type

6. Design credential schema for age verification
7. Build Compact contract for age proof verification
8. Create TypeScript SDK wrapper with clean API
9. Build minimal credential wallet for testing
10. Create demo dApp that integrates the SDK
11. Write developer documentation and integration guide

Deliverable: Functional SDK with age verification, demo app, documentation

Phase 3: Credential Wallet & Additional Verifiers (Weeks 11-16)

Goal: Full credential wallet and expanded verification options

12. Build production credential wallet (Chrome extension or web app)
13. Add token balance verification (prove holding X tokens)
14. Add NFT ownership verification
15. Implement verification request/approval flow
16. Build audit log and verification history
17. Create React component library for developers

Deliverable: Credential wallet, 3+ verification types, React components

Phase 4: Developer Experience & Soft Launch (Weeks 17-22)

Goal: Polish developer experience and onboard early adopters

18. Build developer dashboard (API keys, usage metrics)
19. Create interactive documentation and playground
20. Implement custom policy builder for complex verification rules
21. Launch testnet beta with 3-5 partner dApps
22. Gather feedback and iterate on SDK design
23. Security review and testing

Deliverable: Developer dashboard, beta integrations, refined SDK

Phase 5: Mainnet Launch (Weeks 23-30)

Goal: Production deployment and revenue generation

24. Security audit (required for production identity system)
25. Mainnet deployment
26. Implement billing and usage metering
27. Launch public marketing and developer outreach
28. Onboard paying customers
29. Establish support and maintenance processes

Deliverable: Live production system, paying customers, sustainable operations

Business Model

Revenue Streams

1. Per-Verification API Fees

Primary revenue model—charge developers per successful verification:

- $0.05 - $0.20 per verification (volume discounts)
- Comparable to traditional KYC ($1-5) but much cheaper due to ZK efficiency
- Usage metered, billed monthly

2. Subscription Tiers

- **Free Tier:** 100 verifications/month, basic support, testnet only
- **Starter ($49/mo):** 1,000 verifications, mainnet access, email support
- **Growth ($199/mo):** 10,000 verifications, priority support, custom policies
- **Enterprise (Custom):** Unlimited, SLA, dedicated support, white-label options

3. Future Revenue Opportunities

- **Credential Issuance Fees:** Partner with issuers, take cut of credential creation
- **White-Label Licensing:** Large organizations deploy their own branded version
- **Premium Verifiers:** Higher-value verifications (accredited investor, professional license)

Competitive Advantages

1. **First-Mover on Midnight:** Build the default identity layer before competition arrives
2. **Network Effects:** More users with credentials → more valuable for dApps → more users
3. **Developer-First:** Excellent DX creates switching costs and word-of-mouth
4. **Zero Data Liability:** Unlike traditional KYC, we never hold sensitive data

Risks & Mitigations

Risk	Impact	Mitigation
Midnight mainnet delays	High	Build on testnet, stay engaged with Midnight team
IAMX or competitors dominate	High	Focus on developer experience, niche down initially
ZK proof vulnerabilities	Critical	Rely on Midnight's audited primitives, get SDK audited
Chicken-egg: no users/no dApps	Medium	Seed with own demo credentials, partner early with dApps
Regulatory changes	Medium	Privacy-preserving design is regulation-friendly
Solo dev capacity	Medium	Phased approach, leverage Claude CLI, realistic scope

Success Metrics

Phase 1-2 (Foundation & MVP)

- Successfully deploy identity contracts to testnet
- Complete working SDK with age verification
- Demo dApp successfully integrating SDK

Phase 3-4 (Growth & Beta)

- 3-5 dApps integrating SDK in testnet
- 100+ test credentials created
- Positive developer feedback (NPS > 7)
- Complete documentation with <5 min integration time

Phase 5 (Mainnet)

- 10+ production dApp integrations
- 1,000+ verifications per month
- $500+ monthly recurring revenue
- Zero security incidents

Year 1 Targets

- $5-10K monthly recurring revenue
- 50+ integrated dApps
- 10,000+ monthly verifications
- Recognition as go-to identity solution on Midnight

Immediate Next Steps

1. **Environment Setup:** Install Midnight development stack (Compact compiler, proof server, Lace wallet)
2. **Tutorial Completion:** Work through official counter and bulletin board examples
3. **IAMX Research:** Study their approach, identify differentiation opportunities
4. **Community Engagement:** Join Midnight Discord, introduce project concept
5. **Brand Development:** Finalize name, secure domain, basic visual identity
6. **Architecture Design:** Document credential schema and verification flow

— End of Document —